AWS Solution Architect Training with
AWS Cloud Practitioner Global Certification

Capstone Projects

Trainer: Aravindraj.G- Nminds Academy

AWS Academy Certified Educator

5.Configure Apache Web Server on Linux Instance in AWS

SOWMIGHA K.A

CSE-Artificial Intelligence & Machine Learning,

Sri Eshwar College of Engineering,

Coimbatore.

Objective

EC2 (Elastic Compute Cloud) is a core serviceprovided by Amazon Web Services (AWS) that enables users to rent virtual servers in the cloud to run applications, store data, or handle computational workloads.

An EC2 Linux instance is a virtual machine running the Linux operating system on Amazon Web Services (AWS) EC2 (Elastic Compute Cloud). This allows you to run Windows-based applications, manage Windows Server configurations, and use various Microsoft software (like SQL Server, Apache,FileServer etc.) in the cloud without having to manage physical hardware.

Use Cases for EC2 Linux Instances:

- Web and Application Hosting: Run Apache web servers or other Linux-based web applications in the cloud.

  Database Servers: Host SQL Server databases with EC2 Linux instances, using SSH or EC2 for more control.

- Remote Desktop Applications: Set up SSH (Remote Desktop Services) for remote access to Linux desktops in the cloud.

- Development and Testing: Create development environments with linux OS to test php,MySql or other Linux-based applications.

Used Services in AWS

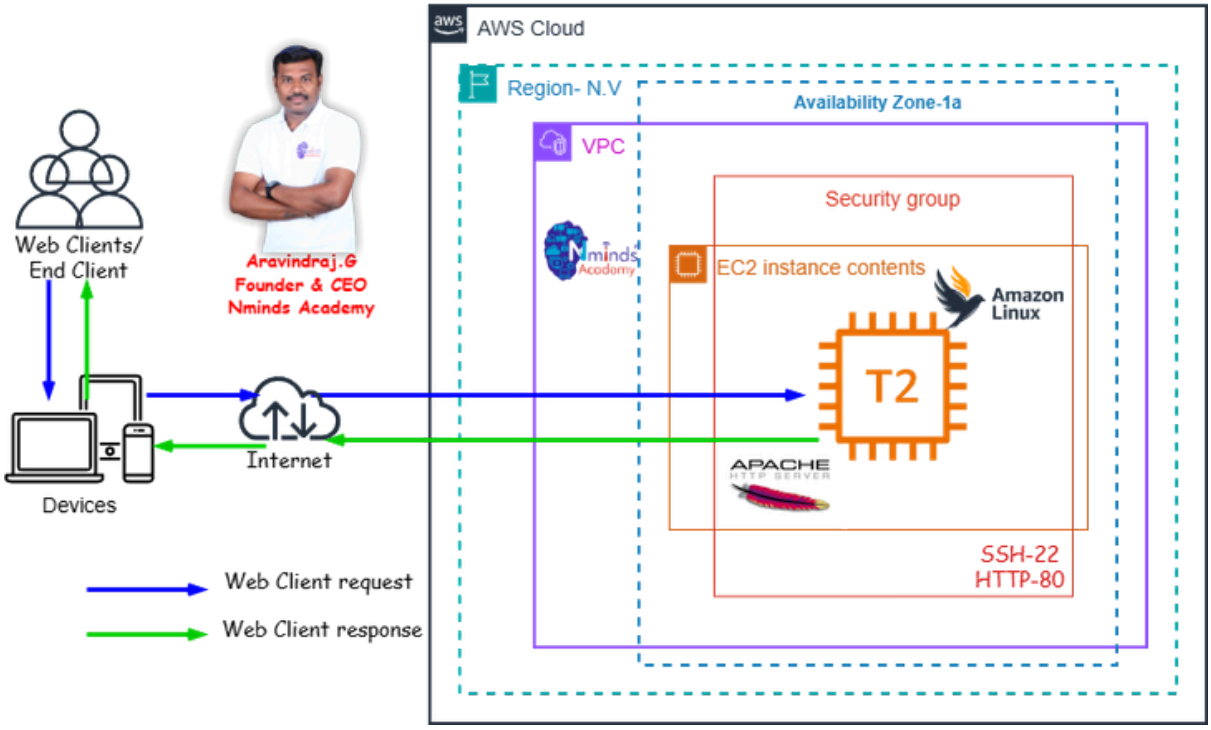
Amazon EC2

Used Services in Linux Server

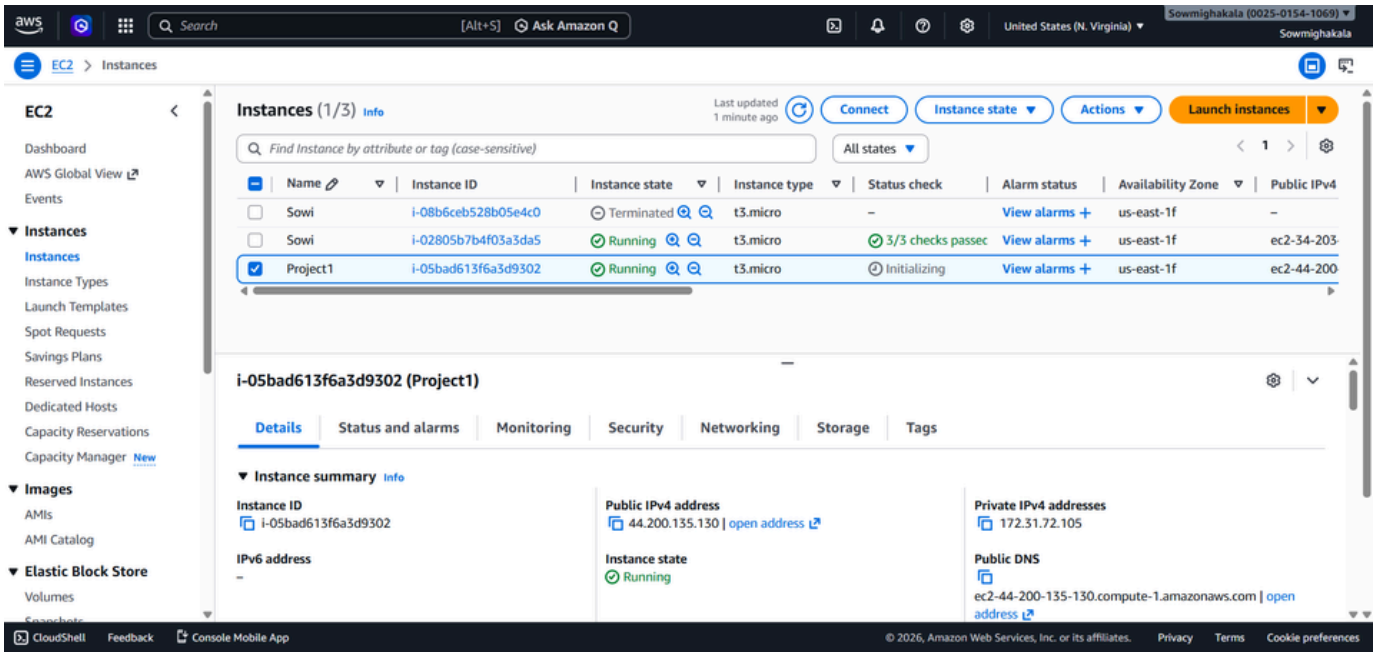
Apache Webserver

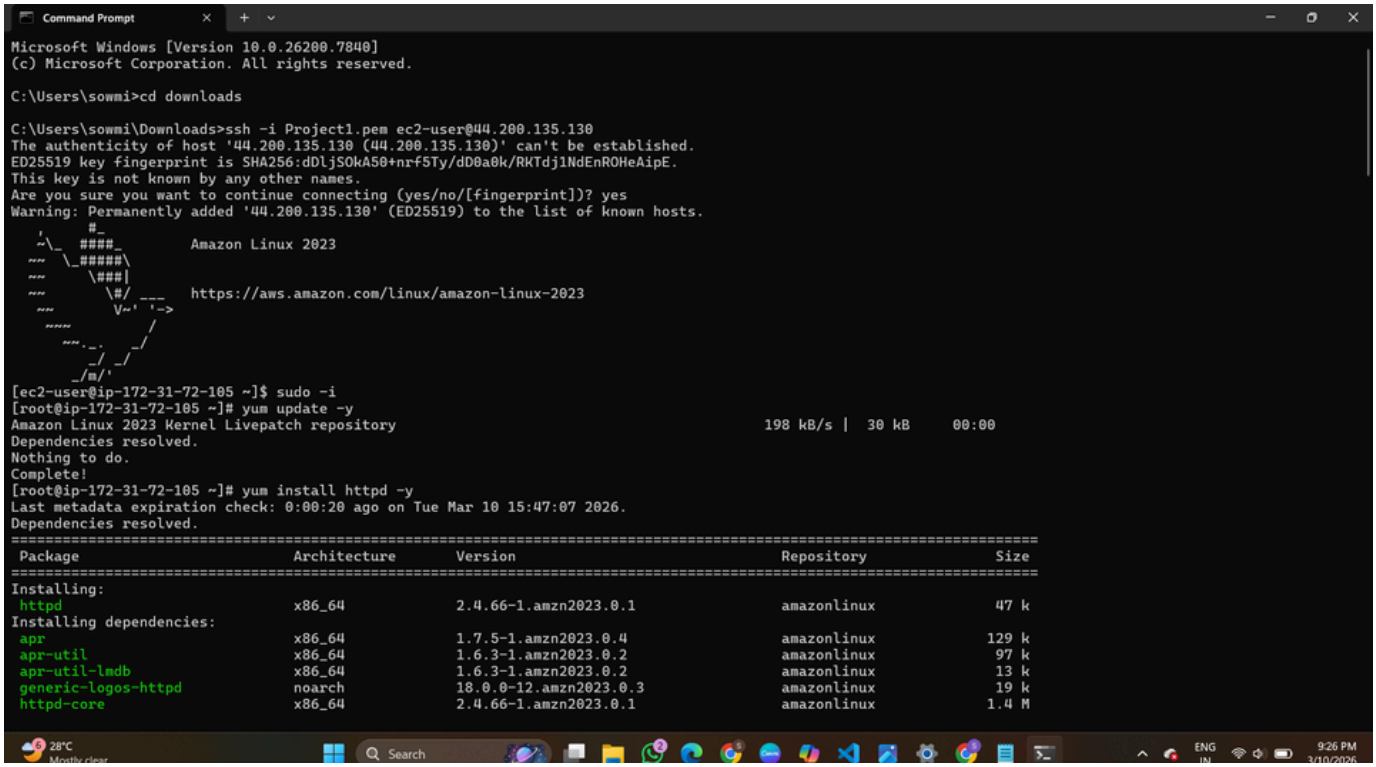
Topology

How to Configure the Apache web server in Linux Server with AWS EC2



Execution Tasks:





Execution Tasks:

```
Command Prompt
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch          httpd-2.4.66-1.amzn2023.0.1.x86_64
  httpd-core-2.4.66-1.amzn2023.0.1.x86_64                    httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
  httpd-tools-2.4.66-1.amzn2023.0.1.x86_64                   libbrotli-1.0.9-4.amzn2023.0.2.x86_64
  mailcap-2.1.49-3.amzn2023.0.3.noarch                       mod_http2-2.0.27-1.amzn2023.0.3.x86_64
  mod_lua-2.4.66-1.amzn2023.0.1.x86_64

Complete!
[root@ip-172-31-72-105 ~]# systemctl status httpd
○ httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:httpd.service(8)
[root@ip-172-31-72-105 ~]# systemctl start httpd
[root@ip-172-31-72-105 ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
     Active: active (running) since Tue 2026-03-10 15:48:09 UTC; 5s ago
       Docs: man:httpd.service(8)
   Main PID: 25423 (httpd)
     Status: "Started, listening on: port 80"
      Tasks: 177 (limit: 1067)
     Memory: 13.4M
        CPU: 87ms
     CGroup: /system.slice/httpd.service
             ├─25423 /usr/sbin/httpd -DFOREGROUND
             ├─25448 /usr/sbin/httpd -DFOREGROUND
             ├─25449 /usr/sbin/httpd -DFOREGROUND
             ├─25461 /usr/sbin/httpd -DFOREGROUND
             └─25478 /usr/sbin/httpd -DFOREGROUND

Mar 10 15:48:09 ip-172-31-72-105.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Mar 10 15:48:09 ip-172-31-72-105.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Mar 10 15:48:09 ip-172-31-72-105.ec2.internal httpd[25423]: Server configured, listening on: port 80
[root@ip-172-31-72-105 ~]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@ip-172-31-72-105 ~]# cd /var/www/html
[root@ip-172-31-72-105 html]# ls
[root@ip-172-31-72-105 html]# vi index.html
"index.html" [New] 113L, 1954B written
[root@ip-172-31-72-105 html]# client_loop: send disconnect: Connection reset
```

```
root@ip-172-31-72-105:~
# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

PubkeyAuthentication no

# The default is to check both .ssh/authorized_keys and .ssh/authorized_keys2
# but this is overridden so installations will only check .ssh/authorized_keys
AuthorizedKeysFile      .ssh/authorized_keys

#AuthorizedPrincipalsFile none


# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# Explicitly disable PasswordAuthentication. By presetting it, we
# avoid the cloud-init set_passwords module modifying sshd_config and
# restarting sshd in the default instance launch configuration.
PasswordAuthentication yes
PermitEmptyPasswords no

# Change to no to disable s/key passwords
#KbdInteractiveAuthentication yes

# Kerberos options
#KerberosAuthentication no
-- INSERT --                                                    65,27         34%
```

Execution Tasks:

It works!

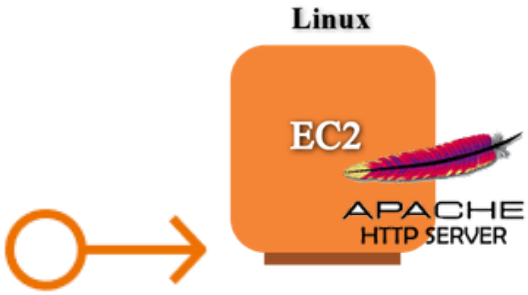
AWS Solution Architect Training with
AWS Cloud Practitioner Global Certification

Capstone Projects

Trainer: Aravindraj.G- Nminds Academy

AWS Academy Certified Educator

6.Configure Elastic IP Address to Windows Web Server in AWS



Aravindraj.G

CSE-Artificial Intelligence & Machine Learning,

Sri Eshwar College of Engineering,

Coimbatore.

Objective

An Elastic IP (EIP) in AWS is a static, public IP address designed for dynamic cloud computing. It is associated with your AWS account and can be quickly associated or disassociated with any EC2 instance in your account. This feature is particularly useful when you need to maintain a consistent IP address for your resources, even when you stop and start EC2 instances.

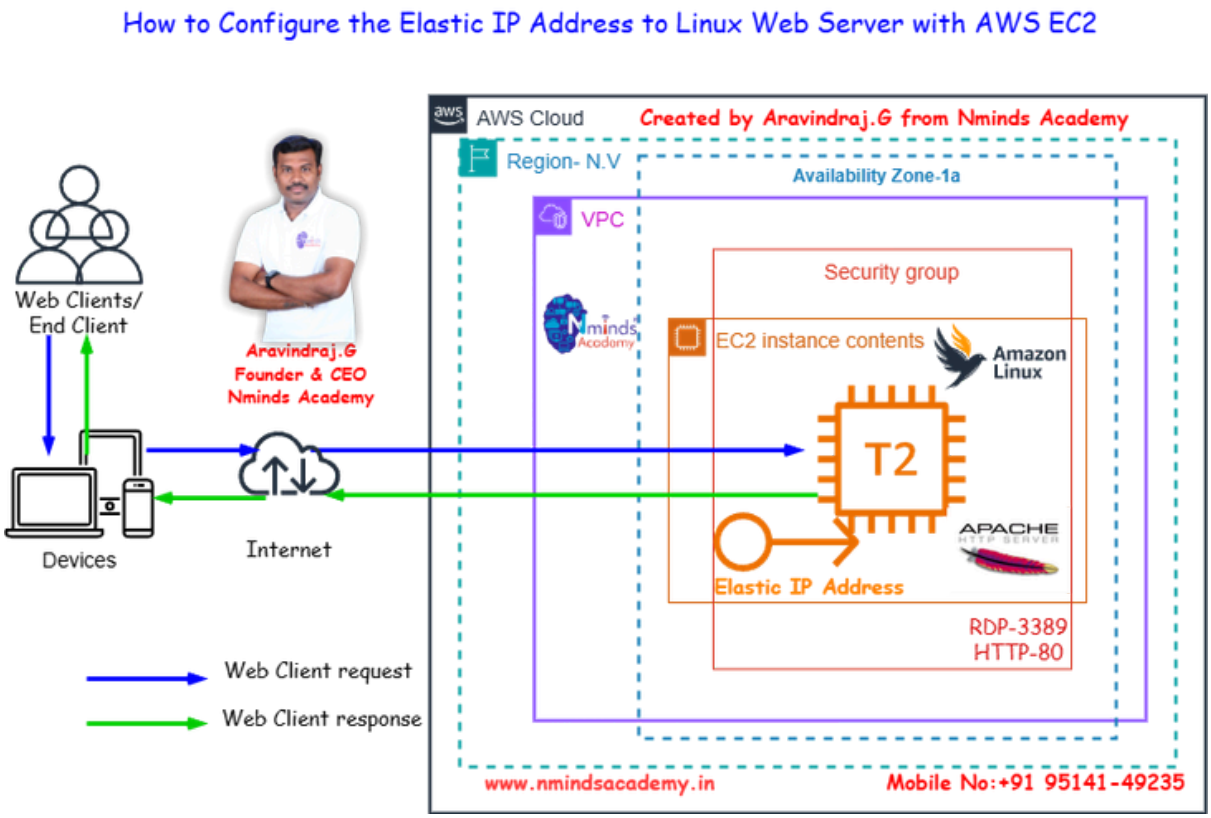
Common Use Cases for Elastic IP:

1. Highly Available Applications:
   - If you're running a service that requires high availability, you can use Elastic IPs to quickly reassign a static IP to a new instance if your primary instance fails, ensuring minimal downtime.
2. Web Servers:
   - If you host a website and need to ensure the IP address remains the same even if the underlying EC2 instance is restarted, an Elastic IP helps maintain this consistency.
3. Disaster Recovery:
   - Elastic IPs are useful for disaster recovery scenarios. If one instance goes down, you can quickly associate the EIP with a backup instance to ensure services are still accessible.

Best Practices:

- Use Elastic IPs only when necessary: Since they come with associated costs when unused, it's a good practice to release EIPs that are no longer required.
- Move IPs during instance failure: Instead of keeping an EIP permanently attached, use it as a failover method, reassociating it to a new instance when needed.
- Monitor EIP Usage: Periodically review your usage to ensure you're not paying for unnecessary Elastic IP addresses.

Topology



Used Services in AWS

Used Services in Windows Server

Amazon EC2

APACHE HTTP SERVER

Apache Webserver

Amazon ElasticIPAddress

Execution Tasks:

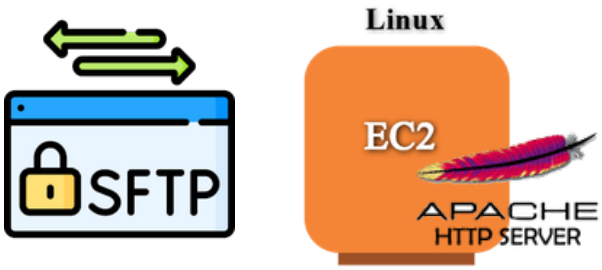
AWS Solution Architect Training with
AWS Cloud Practitioner Global Certification

Capstone Projects

Trainer: Aravindraj.G- Nminds Academy

AWS Academy Certified Educator

7.Configure SFTP toLinux Web Server in AWS



Aravindraj.G

CSE-Artificial Intelligence & Machine Learning,

Sri Eshwar College of Engineering,

Coimbatore.

Objective

EC2 (Elastic Compute Cloud) is a core service provided by Amazon Web Services (AWS) that enables users to rent virtual servers in the cloud to run applications, store data, or handle computational workloads.

Use Cases for EC2 Linux Instances:

- Web and Application Hosting: Run Apache web servers or other Linux-based web applications in the cloud.

  Database Servers: Host SQL Server databases with EC2 Linux instances, using SSH or EC2 for more control.

- Remote Desktop Applications: Set up SSH (Remote Desktop Services) for remote access to Linux desktops in the cloud.

- Development and Testing: Create development environments with linux OS to test php,MySql or other Linux-based applications.

SFTP- Secure File Transfer Protocol

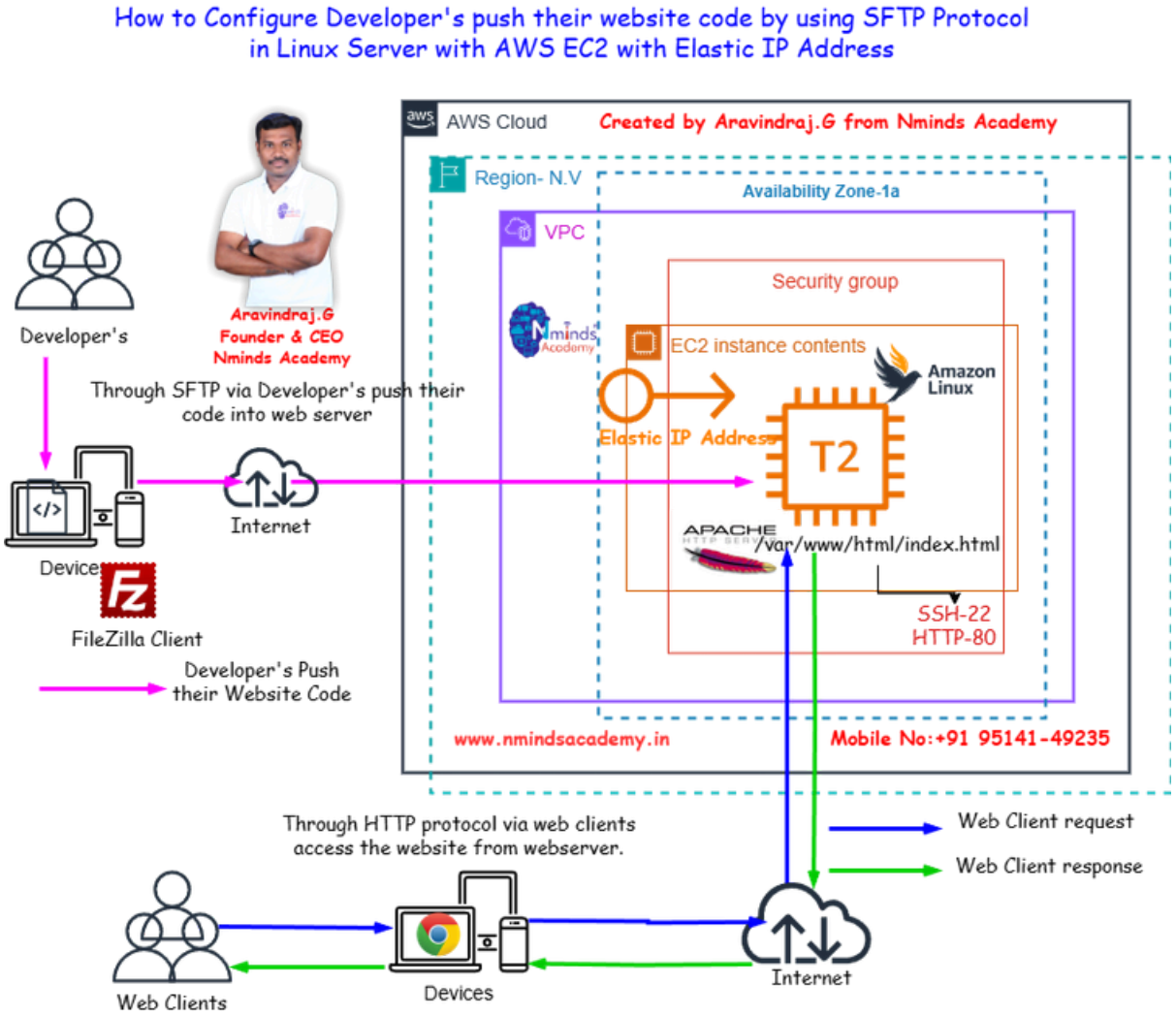
SFTP (Secure File Transfer Protocol) is a secure network protocol that allows users to transfer files over a secure, encrypted connection. It is a more secure alternative to FTP (File Transfer Protocol) and is commonly used to upload, download, and manage files on remote servers.

Here are the key uses and advantages of SFTP:

1. File Transfer: SFTP allows secure transfer of files between a local and a remote computer over a network. It ensures that the data is encrypted during transmission, protecting it from eavesdropping or tampering.
2. File Management: Besides transferring files, SFTP also allows users to perform file management tasks such as renaming, deleting, and moving files on the remote server.
3. Secure Authentication: SFTP uses SSH (Secure Shell) for authentication, which provides a stronger level of security compared to FTP's traditional username/password system. It often supports public key authentication, adding an extra layer of security.
4. Encryption: The main advantage of SFTP over FTP is encryption. All data sent over SFTP is encrypted, ensuring that sensitive information (like passwords and files) is protected during transmission.

5. Firewall-Friendly: SFTP operates over a single port (usually port 22), making it easier to use behind firewalls and NAT (Network Address Translation) devices compared to FTP, which uses multiple ports.

Topology



Used Services in AWS

Used Services in Windows Server

Amazon EC2

APACHE HTTP SERVER

Apache Webserver

Amazon Elastic IP Address

Execution Tasks: